1-

```
module Dynamic_arr_tb();
    // declare two dynamic arrays dyn_arr1, dyn_arr2 of type int
    int  dyn_arr1 [], dyn_arr2 [];

    initial begin
        // initialize dyn_arr2 array elements with (9,1,8,3,4,4)
        dyn_arr2 = new[6];
        dyn_arr2 = '{9,1,8,3,4,4};

        // allocate six elements in array dyn_arr1 using new operator
        dyn_arr1 = new[6];

        // initialize array dyn_arr1 with index as its value using foreach
        foreach(dyn_arr1[i])
            dyn_arr1[i] = i;

        // display dyn_arr1 and its size
        foreach(dyn_arr1[i])
            $display("dyn_arr1[%0d] = %0d",i,dyn_arr1[i]);

        $display("dyn_arr1.size = %0d",dyn_arr1.size());
        // delete array dyn_arr1
        dyn_arr1.delete();

        //reverse, sort, reverse sort and shuffle the array dyn_arr2 and display dyn_arr2 after using each method
        dyn_arr2.reverse();
        $display("dyn_arr2.reverse = %p",dyn_arr2);

        dyn_arr2.sort();
        $display("dyn_arr2.sort = %p",dyn_arr2);

        dyn_arr2.rsort();
        $display("dyn_arr2.rsort = %p",dyn_arr2);

        for (int i =0 ; i < 3; i = i + 1) begin
            dyn_arr2.shuffle();
            $display("iteration: %0d --> dyn_arr2 = %p",i,dyn_arr2);
        end
    end

endmodule
```

```
# vsim work.Dynamic_arr_tb
# Start time: 17:45:15 on Mar 24,2026
# ** Note: (vsim-3812) Design is being optimized...
# Loading sv_std.std
# Loading work.Dynamic_arr_tb(fast)
# dyn_arr1[0] = 0
# dyn_arr1[1] = 1
# dyn_arr1[2] = 2
# dyn_arr1[3] = 3
# dyn_arr1[4] = 4
# dyn_arr1[5] = 5
# dyn_arr1.size = 6
# dyn_arr2.reverse = '{4, 4, 3, 8, 1, 9}
# dyn_arr2.sort = '{1, 3, 4, 4, 8, 9}
# dyn_arr2.rsort = '{9, 8, 4, 4, 3, 1}
# iteration: 0 --> dyn_arr2 = '{8, 1, 4, 3, 9, 4}
# iteration: 1 --> dyn_arr2 = '{1, 4, 9, 3, 4, 8}
# iteration: 2 --> dyn_arr2 = '{3, 8, 9, 1, 4, 4}
```

2-

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
COUNTER_1	When the reset is asserted, the output counter value should be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
COUNTER_2	When the load is asserted, the output count_out should take the value of the load_data input	Randomization under Constraint the load signal to be active 70% of the time	-	A checker in the testbench to make sure the output is correct
COUNTER_3	When both ce and up_down are asserted to high, the output count_out should increment by 1	Randomization for up_down and Randomization under Constraint the enable signal to be active 70% of the time	-	A checker in the testbench to make sure the output is correct
COUNTER_4	When the ce is asserted and up_down is asserted to low, the output count_out should decrement by 1	Randomization for up_down and Randomization under Constraint the enable signal to be active 70% of the time	-	A checker in the testbench to make sure the output is correct

counter_pkg.sv

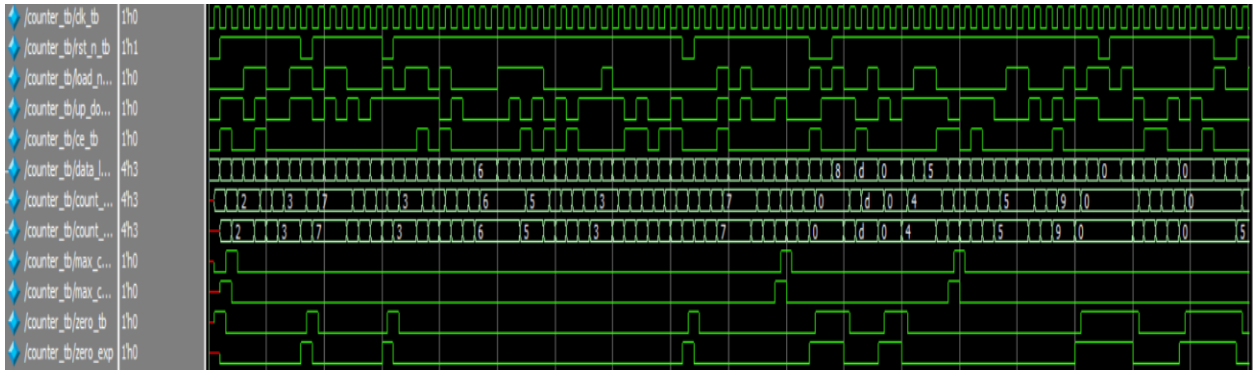
```
package counter_pkg;

    parameter COUNTER_WIDTH = 4;

    class counter_class;
        rand bit rst_n, load_n, up_down, ce;
        rand bit [COUNTER_WIDTH-1:0] data_load;

        constraint rst_cst{
            rst_n dist {0:= 5, 1:= 95};
        }

        constraint load_cst{
            load_n dist{0:= 70, 1:= 30};
        }

        constraint ce_cst{
            ce dist{0:= 70, 1:= 30};
        }

    endclass

endpackage
```

counter_tb.sv

```
import counter_pkg::*;

module counter_tb();

    counter_class stimulus = new();

    //1 declare local wire and reg
    bit clk_tb, rst_n_tb, load_n_tb, up_down_tb, ce_tb;
    bit [COUNTER_WIDTH-1:0] data_load_tb;
    logic [COUNTER_WIDTH-1:0] count_out_tb, count_out_exp;
    logic max_count_tb, max_count_exp;
    logic zero_tb, zero_exp;

    int error_count, correct_count;

    //2 Instantiate the module under test
    counter #(.WIDTH(COUNTER_WIDTH)) uut(
            clk_tb,
            rst_n_tb,
            load_n_tb,
            up_down_tb,
            ce_tb,
            data_load_tb,
            count_out_tb,
            max_count_tb,
            zero_tb);

    //3 General stimuli using always and initial
    initial begin
        clk_tb = 0;
        forever #5 clk_tb = ~clk_tb; // Generate a clock signal with a period of 10 time units
    end
```

```
    // Stimulus test
    initial begin
        assert_reset();
        repeat(10000) begin
            assert (stimulus.randomize());
            rst_n_tb = stimulus.rst_n;
            load_n_tb = stimulus.load_n;
            ce_tb = stimulus.ce;
            up_down_tb = stimulus.up_down;
            data_load_tb = stimulus.data_load;
            check_result(stimulus);
        end
        $display("Total Correct: %0d, Total Errors: %0d", correct_count, error_count);
        $stop;
    end

    // Task to assert reset
    task assert_reset;
        rst_n_tb = 1'b0;
        check_reset;
        rst_n_tb = 1'b1;
    endtask

    // Task to check reset behavior
    task check_reset;
        @(negedge clk_tb);
        if ((count_out_tb !== 0) && (zero_tb !== 1) && (max_count_tb !== 0)) begin
            error_count += 1;
            $display("%t: ERROR for asserting reset--> count_out_tb = %0d, zero_tb = %0d, max_count_tb = %0d",
                    $time, count_out_tb, zero_tb, max_count_tb);
        end else begin
            correct_count += 1;
            $display("%t: SUCCESS for asserting reset--> count_out_tb = %0d, zero_tb = %0d, max_count_tb = %0d",
                    $time, count_out_tb, zero_tb, max_count_tb);

        end
    endtask
```

```
    // Task to implement the golden model and check results
    task golden_model(input counter_class stimulus_c);
        if (!stimulus_c.rst_n)
            count_out_exp = 0;
        else if (!stimulus_c.load_n)
            count_out_exp = data_load_tb;
        else if (stimulus_c.ce)
            if (stimulus_c.up_down)
                count_out_exp = count_out_exp + 1;
            else
                count_out_exp = count_out_exp - 1;
        max_count_exp = (count_out_exp == {COUNTER_WIDTH{1'b1}})? 1:0;
        zero_exp = (count_out_exp == 0)? 1:0;
    endtask

    // Task to check results against the golden model
    task check_result(input counter_class stimulus_t);
        golden_model(stimulus_t);
        if(stimulus_t.rst_n !== 0) begin
            @(negedge clk_tb);
            if ((count_out_tb !== count_out_exp) || (zero_tb !== zero_exp) || (max_count_tb !== max_count_exp)) begin
            error_count += 1;
            $display("%t: ERROR--> count_out_tb = %0d,count_out_exp = %0d, zero_tb = %0d, zero_exp = %0d, max_count_tb = %0d,max_count_exp = %0d",
                    $time,count_out_tb, count_out_exp, zero_tb, zero_exp, max_count_tb, max_count_exp);
            end
            else begin
                correct_count += 1;
                $display("%t: SUCCESS--> count_out_tb = %0d,count_out_exp = %0d, zero_tb = %0d, zero_exp = %0d, max_count_tb = %0d,max_count_exp = %0d",
                        $time,count_out_tb, count_out_exp, zero_tb, zero_exp, max_count_tb, max_count_exp);
            end
        end else begin
            check_reset;
        end
    endtask

endmodule
```




```
Total Correct: 10001, Total Errors: 0
** Note: $stop    : counter_tb.sv(47)
   Time: 100010 ns  Iteration: 1  Instance: /counter_tb
```

```
================================================================================
=== Instance: /counter_tb/uut
=== Design Unit: work.counter
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        10        10         0   100.00%

================================Branch Details================================
```

```
Condition Coverage:
    Enabled Coverage              Bins   Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Conditions                       2         2         0   100.00%

================================Condition Details================================
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       7         7         0   100.00%

================================Statement Details================================
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         30        30         0   100.00%

================================Toggle Details================================
```

3-

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALSU_1	When the reset is asserted, the outputs should be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
ALSU_2	Incase of invalid cases occur, Leds should blink and out bits are set to low,	Randomization under constraint where invalid cases shouldn't occur as frequent as valid cases	-	A checker in the testbench to make sure the output is correct
ALSU_3	Incase of invalid cases do not occur,a and the bypass_A or bypass_B are high then the output will take the value of A or B depending on the parameter INPUT_PRIORITY	Randomization under constraint on bypass ports are disabled most of the time		
ALSU_4	Incase of invalid cases do not occur, when opcode is ADD, then out should perform the addition on ports A and B taking cin if parameter FULL_ADDER is high	Randomization Constraint for adder inputs (A, B) to take the values (MAXPOS, ZERO and MAXNEG) more often than the other values	-	A checker in the testbench to make sure the output is correct
ALSU_5	Incase of invalid cases do not occur, when opcode is MULT, then out should perform the multiplication on ports A and B	Randomization Constraint for adder inputs (A, B) to take the values (MAXPOS, ZERO and MAXNEG) more often than the other values	-	A checker in the testbench to make sure the output is correct
ALSU_6	Incase of invalid cases do not occur, when opcode is OR, then out should perform the OR operation on ports A and B if reduction_A and reduction_B are low	Randomization without constraint on A or B.	-	A checker in the testbench to make sure the output is correct
ALSU_7	Incase of invalid cases do not occur, when opcode is OR and if reduction_A or reduction_B is high, then output should perform the reduction OR operation on ports A or ports B depends on the priority	Randomization uunder the constraint on the input A most of the time to have one bit high in its 3 bits while constraining the B bits to be low if reduction_A is high	-	A checker in the testbench to make sure the output is correct
ALSU_8	Incase of invalid cases do not occur, when opcode is OR, then out should perform the OR operation on ports A and B if reduction_A and reduction_B are low	Randomization without constraint on A or B.	-	A checker in the testbench to make sure the output is correct
ALSU_9	Incase of invalid cases do not occur, when opcode is XOR and if reduction_A or reduction_B is high, then output should perform the reduction OR operation on ports A or ports B depends on the priority	Randomization uunder the constraint on the input A most of the time to have one bit high in its 3 bits while constraining the B bits to be low if reduction_A is high	-	A checker in the testbench to make sure the output is correct
ALSU_10	Incase of invalid cases do not occur, when opcode is SHIFT, then out should be shifted by 1 bit depending on direction bit	Randomization without constraint	-	A checker in the testbench to make sure the output is correct
ALSU_11	Incase of invalid cases do not occur, when opcode is ROTATE, then out should be rotated by 1 bit depending on direction bit	Randomization without constraint	-	A checker in the testbench to make sure the output is correct

```
package ALSU_pkg;

    typedef enum bit [2:0] { OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID6, INVALID7 } opcode_t;
    typedef enum bit [2:0] { ZERO = 3'b000, MAXPOS = 3'b011, MAXNEG = 3'b100} reg_e;

    class ALSU_cst;
        rand logic cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
        rand opcode_t opcode;
        rand logic [2:0] A, B;
        rand reg_e A_val, B_val;
        rand bit [2:0] A_val_ext, B_val_ext;
        bit [2:0] walking_ones[] = '{3'b100, 3'b010, 3'b001};
        rand bit [2:0] walking_ones_t, walking_ones_f;

        //1. Reset to be asserted with a low probability that you decide.
        constraint rst_cst{
            rst dist{0:= 99, 1:= 1};
        }
```

```
        //2. The inputs A and B to be mostly zero, with some random values.
        constraint A_B_cst{
            walking_ones_t inside{walking_ones}; // Constrain the walking_ones_t to be one of the patterns in walking_ones
            !(walking_ones_f inside {walking_ones}); // Constrain the walking_ones_f to not be any of the patterns in walking_ones
            !(A_val_ext inside {A_val}); // Constrain A_val_ext to not be any of the values in A_val_ext (which is effectively no constraint since it's a random variable)
            !(B_val_ext inside {B_val}); // Constrain B_val_ext to not be any of the values in B_val_ext (which is effectively no constraint since it's a random variable)

            // If the opcode is ADD or MULT, constraint the inputs A and B  to take the values (MAXPOS, ZERO and MAXNEG) with higher probability than the other values
            if ((opcode == ADD) || (opcode == MULT)){
                red_op_A dist {1:= 10, 0 := 90};
                red_op_B dist {1:= 10, 0 := 90};
                A dist {A_val := 90, A_val_ext := 10};
                B dist {B_val := 90, B_val_ext := 10};
            }
            else {
                // If the opcode is OR or XOR
                if ((opcode == OR) || (opcode == XOR)){
                    // red_op_A is high, constraint the input A most of the time
                    // to have one bit high in its 3 bits while constraining the B bits to be low
                    if (red_op_A == 1){
                        A dist {walking_ones := 90, walking_ones_f := 10};
                        B == 3'b000;
                    }
                    // red_op_B is high, constraint the input B most of the time
                    // to have one bit high in its 3 bits while constraining the A bits to be low
                    else if (red_op_B == 1){
                        B dist {walking_ones := 90, walking_ones_f := 10};
                        A == 3'b000;
                    }
                    // Do not constraint the inputs A or B when the operation is shift or rotate
                    else{
                        // Do nothing
                    }
                }
            }
        }
```

```
        // 3. The opcode to be mostly OR, XOR, ADD, MULT, ROTATE, SHIFT, with some random values for the other operations.
        constraint opcode_cst{
            opcode dist{[OR:ROTATE] := 90, [INVALID6:INVALID7] := 10};
        }

        // 4. The bypass signals to be mostly disabled, with some random values.
        constraint bypass_cst{
            bypass_A dist{0:= 95, 1:= 5};
            bypass_B dist{0:= 95, 1:= 5};
        }

    endclass

endpackage
```

```
import ALSU_pkg::*;

module ALSU_tb;

    // 1- declare parameter, classes, local wire and reg
    ALSU_cst ALSU_obj = new();

    parameter INPUT_PRIORITY = "A";
    parameter FULL_ADDER = "ON";
    bit clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
    opcode_t opcode;
    bit  signed [2:0] A, B;
    logic [15:0] leds;
    bit [15:0] leds_exp;
    logic signed [5:0] out;
    bit signed [5:0] out_exp;
    bit signed [5:0] prev_out_exp;

    int error_count = 0;
    int correct_count = 0;

    // 2- Instantiate the module under test
        ALSU #(.INPUT_PRIORITY(INPUT_PRIORITY), .FULL_ADDER(FULL_ADDER)) uut(
        .clk(clk),
        .cin(cin),
        .rst(rst),
        .red_op_A(red_op_A),
        .red_op_B(red_op_B),
        .bypass_A(bypass_A),
        .bypass_B(bypass_B),
        .direction(direction),
        .serial_in(serial_in),
        .opcode(opcode),
        .A(A),
        .B(B),
        .leds(leds),
        .out(out)
    );

    // 3- Generate the clock
    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end
```

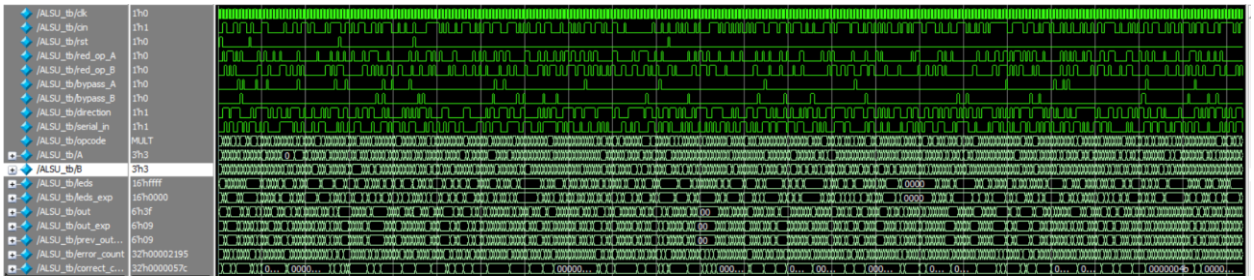
```
// 4- General stimuli
initial begin
    #5;
    // assert reset
    assert_reset;

    repeat(10000) begin
        assert(ALSU_obj.randomize());
        cin = ALSU_obj.cin;
        rst = ALSU_obj.rst;
        red_op_A = ALSU_obj.red_op_A;
        red_op_B = ALSU_obj.red_op_B;
        bypass_A = ALSU_obj.bypass_A;
        bypass_B = ALSU_obj.bypass_B;
        direction = ALSU_obj.direction;
        serial_in = ALSU_obj.serial_in;
        opcode = ALSU_obj.opcode;
        A = ALSU_obj.A;
        B = ALSU_obj.B;

        check_result(ALSU_obj);
    end

    $display("Total Correct: %0d, Total Errors: %0d", correct_count, error_count);
    $stop;
end

// Task to assert reset
task assert_reset;
    rst = 1'b1;
    @(negedge clk);
    check_reset;
    rst = 1'b0;
endtask
```

```
// Task to check reset behavior
task check_reset;
    @(negedge clk);
    if ((leds !== 0) || (out !== 0)) begin
        error_count += 1;
        $display("%t: ERROR for asserting reset--> leds = %0d, out = %0d",
                $time, leds, out);
    end else begin
        correct_count += 1;
        $display("%t: SUCCESS for asserting reset--> leds = %0d, out = %0d",
                $time, leds, out);
    end
endtask

// function to check validation
function bit is_invalid(input ALSU_cst ALSU_obj1);
    bit valid;
    if((ALSU_obj1.red_op_A || ALSU_obj1.red_op_B) && (ALSU_obj1.opcode[1] | ALSU_obj1.opcode[2]))
        valid = 1'b1;
    else if (ALSU_obj1.opcode[1] & ALSU_obj1.opcode[2])
        valid = 1'b1;
    else
        valid = 1'b0;
    return valid;
endfunction
```

```
// Task to implement the golden model and check results
task golden_model (input ALSU_cst ALSU_obj2);
    if (ALSU_obj2.rst == 1'b1) begin
        leds_exp = 0;
        out_exp = 0;
        prev_out_exp = 0;
    end else begin
        if (is_invalid(ALSU_obj2)) begin
            leds_exp = 16'hFFFF;
        end
        else begin
            leds_exp = 0;
        end
    end

    if (ALSU_obj2.bypass_A && ALSU_obj2.bypass_B)
         out_exp = (INPUT_PRIORITY == "A")? A: B;
    else if (ALSU_obj2.bypass_A)
        out_exp = A;
    else if (ALSU_obj2.bypass_B)
        out_exp = B;
    else if (is_invalid(ALSU_obj2))
        out_exp = 0;
    else begin
        case (ALSU_obj2.opcode)
        OR: begin
            if (ALSU_obj2.red_op_A && ALSU_obj2.red_op_B)
            out_exp = (INPUT_PRIORITY == "A")? |A: |B;
            else if (ALSU_obj2.red_op_A)
            out_exp = |A;
            else if (ALSU_obj2.red_op_B)
            out_exp = |B;
            else
            out_exp = A | B;
        end
```

```
            XOR: begin
                if (ALSU_obj2.red_op_A && ALSU_obj2.red_op_B)
                out_exp = (INPUT_PRIORITY == "A")? ^A: ^B;
                else if (ALSU_obj2.red_op_A)
                out_exp = ^A;
                else if (ALSU_obj2.red_op_B)
                out_exp = ^B;
                else
                out_exp = A ^ B;
            end
            ADD: begin
                if (FULL_ADDER == "ON") begin
                out_exp = A + B + cin;
                end else begin
                out_exp = A + B;
                end
            end
            MULT: out_exp = A * B;
            SHIFT: begin
                if (ALSU_obj2.direction)
                out_exp = {prev_out_exp[4:0], ALSU_obj2.serial_in};
                else
                out_exp = {ALSU_obj2.serial_in, prev_out_exp[5:1]};
            end
            ROTATE: begin
                if (ALSU_obj2.direction)
                out_exp = {prev_out_exp[4:0], prev_out_exp[5]};
                else
                out_exp = {prev_out_exp[0], prev_out_exp[5:1]};
            end
            default: out_exp = 0;
            endcase
        end
        prev_out_exp = out_exp; // Update prev_out_exp for the next cycle
    endtask
```

```
    // Task to check results against the golden model
    task check_result(input ALSU_cst ALSU_obj3);
        golden_model(ALSU_obj3);
        if(!ALSU_obj3.rst) begin
            @(negedge clk);
            if ((out_exp !== out) || (leds_exp !== leds)) begin
            error_count += 1;
            $display("%t: ERROR--> out = %0d,out_exp = %0d, leds = %0d, leds_exp = %0d",
                    $time,out, out_exp, leds, leds_exp);
            end
            else begin
                correct_count += 1;
                $display("%t: SUCCESS--> out = %0d,out_exp = %0d, leds = %0d, leds_exp = %0d",
                        $time,out, out_exp, leds, leds_exp);
            end
        end else begin
            check_reset;
        end
    endtask


endmodule
```




```
=================================================================================
=== Instance: /ALSU_tb/uut
=== Design Unit: work.ALSU
=================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        31        31         0   100.00%

=================================Branch Details=================================
```

```
Condition Coverage:
    Enabled Coverage              Bins   Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Conditions                       6         6         0   100.00%

=================================Condition Details================================
```

```
Expression Coverage:
    Enabled Coverage              Bins   Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Expressions                      8         8         0   100.00%

=================================Expression Details================================
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      48        48         0   100.00%

=================================Statement Details================================
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                        116       116         0   100.00%

=================================Toggle Details================================
```